HW04 — Neural Networks

Specification

Ramaz ML Course 2026

This document is the **contract** for HW04. It tells you exactly what to build, what each function and method must do, and the shape of the public API your code must match. The test suite checks this contract directly — when your code matches the spec, the tests pass.

How to use this document: Read it once end-to-end before opening any of the Python files. Then implement in roughly the order tests are written: `network.py` first (Dropout, then MLP), then `training.py`, then `analysis.py`. Refer back here whenever you forget a math formula or a return shape.

All derivations referenced below live in the Lesson 5 LaTeX notes. The spec points to them rather than re-deriving everything.

Contents

1 Architecture overview **2**

2 `Dropout` (`network.py`) **2**
- 2.1 `__init__` . . . 2
- 2.2 `forward` . . . 2

3 `MLP` (`network.py`) **3**
- 3.1 `__init__` . . . 3
- 3.2 `forward` . . . 4

4 `train_one_epoch` (`training.py`) **4**

5 `validate` (`training.py`) **5**

6 `train` (`training.py`) **5**

7 Analysis helpers (`analysis.py`) **6**
- 7.1 `load_fashion_mnist` . . . 6
- 7.2 `build_model` . . . 7

8 How to read the test file **7**

9 Restrictions **8**

1 Architecture overview

HW04 has three implementation files plus an analysis script:

`network.py`	`Dropout` (custom nn.Module), `MLP` (configurable feedforward net)
`training.py`	`train_one_epoch`, `validate`, `train` (orchestrator)
`analysis.py`	`load_fashion_mnist`, `build_model` (you implement); teacher-provided `__main__` block runs the ablation

You will implement everything in those three files *above* the teacher-provided separator in `analysis.py`. The training run for Part 2 of the assignment is handled by the `if __name__ == "__main__":` block in `analysis.py`, which is released as-is.

The high-level flow is the canonical PyTorch training loop you saw in lecture:

1. `build_model(config)` returns the model, optimizer, and (optional) scheduler.
2. `load_fashion_mnist` returns three `DataLoader`s.
3. `train(...)` loops over epochs, calling `train_one_epoch` and `validate`, applying early stopping, restoring the best snapshot.
4. The orchestration block plots curves and prints a results table.

2 `Dropout` (`network.py`)

The first thing you implement. `Dropout` is a regularization layer that zeros activations at random during training, scaling the survivors so the expected activation is unchanged. At evaluation time it does nothing.

2.1 `__init__`

```
def __init__(self, p:  float) -> None
```

Construct a fresh `Dropout` layer with rate p. Subclass `torch.nn.Module`; remember to call `super().__init__()` before anything else.

Store:

- `self.p` — the dropout probability, as-is.

The tests read `self.p` after construction, so use this exact attribute name. `p` must lie in $[0, 1)$. If `p == 0`, the layer is the identity in both modes.

2.2 `forward`

```
def forward(self, x:  torch.Tensor) -> torch.Tensor
```

Apply (or skip) inverted dropout depending on the layer's mode. PyTorch sets `self.training` to `True` after `model.train()` and `False` after `model.eval()` — read this flag inside `forward` to decide which branch to take.

The **inverted-dropout** rule (see Lesson 5, Session B, Section "Dropout" for the derivation):

$$\tilde{x}_i = \begin{cases} \dfrac{x_i}{1-p} & \text{with probability } 1-p \\ 0 & \text{with probability } p \end{cases}$$

when `self.training` is `True` and $p > 0$; otherwise $\tilde{x} = x$ (the identity).

The Bernoulli mask is sampled independently per element. Use `torch.rand_like(x)` as the source of randomness; the autouse seed fixture in `conftest.py` makes this reproducible across test runs.

- Input shape: arbitrary (Dropout is shape-preserving).
- Output shape: identical to the input.
- Expectation: $\mathbb{E}[\tilde{x}] = x$ across the random mask.

Invariants the tests will check

- In training mode, with $p = 0.3$ and a large input, the fraction of zero entries in the output is approximately 0.3.
- In training mode, every non-zero entry equals $x_i/(1-p)$.
- In eval mode, `Dropout(x)` equals `x` exactly.
- Switching back from `eval()` to `train()` restores dropping behavior.
- `Dropout` is an `nn.Module` subclass.

3 MLP (`network.py`)

A multi-layer perceptron with configurable depth, hidden widths, and two optional regularizers. The architecture is exactly the one in Lesson 5 Session A.

3.1 `__init__`

```
def __init__(self, input_dim:  int, hidden_dims:  list[int],
    output_dim:  int, dropout_p:  float = 0.0, use_batchnorm:  bool = False) -> None
```

Subclass `nn.Module` (remember `super().__init__()`). Build the following architecture, in order:

1. One `nn.Flatten` (so 2-D image inputs like FashionMNIST collapse to a single feature dimension).
2. For each entry h in `hidden_dims`, in order:
   (a) A `nn.Linear` from the previous dimension to h.
   (b) If `use_batchnorm` is `True`: an `nn.BatchNorm1d(h)`.
   (c) An `nn.ReLU`.
   (d) If `dropout_p > 0`: your custom `Dropout(p=dropout_p)`.
3. One final `nn.Linear` from the last hidden dimension to `output_dim`.

The final layer produces **raw logits** — do not apply softmax or any activation. The loss function (`nn.CrossEntropyLoss`) applies the softmax internally.

Register the layers as submodules so that `model.parameters()` returns all of them. The simplest way is to wrap everything in an `nn.Sequential` and store it as an attribute.

3.2 `forward`

```
def forward(self, x:  torch.Tensor) -> torch.Tensor
```

Run `x` through every layer in order: `nn.Flatten`, then the hidden blocks, then the final linear projection. Return the resulting logit tensor.

- Input shape: `(B, 1, 28, 28)` for FashionMNIST, but the method must accept any input that `nn.Flatten` reduces to shape `(B, input_dim)`.
- Output shape: `(B, output_dim)`.

Invariants the tests will check

- `MLP` is an `nn.Module` subclass.
- Layer counts match the architecture above:
  - Exactly one `nn.Flatten`.
  - `len(hidden_dims) + 1` `nn.Linear` layers.
  - `len(hidden_dims)` `nn.ReLU` layers.
  - `len(hidden_dims)` `nn.BatchNorm1d` layers when `use_batchnorm`, zero otherwise.
  - `len(hidden_dims)` `Dropout` layers when `dropout_p > 0`, zero otherwise.
- Each `nn.Linear`'s `in_features` / `out_features` match the adjacent dimensions.
- `forward` produces `(B, output_dim)` for inputs of shape `(B, 1, 28, 28)`, for $B \in \{1, 4, 32\}$.

4 `train_one_epoch` (`training.py`)

```
def train_one_epoch(model:  nn.Module, loader:  DataLoader,
    criterion:  nn.Module, optimizer:  torch.optim.Optimizer,
    device:  torch.device | str) -> float
```

Run one full pass over `loader`, updating model parameters. Return the mean per-example training loss across the epoch.

Algorithm (one iteration of the standard PyTorch training loop):

1. Switch the model to training mode (so `Dropout` and `BatchNorm1d` behave correctly).
2. For each `(x, y)` batch in `loader`:
   (a) Move the batch tensors onto `device`.
   (b) Zero the optimizer's accumulated gradients.
   (c) Feed the batch through the model to obtain its logit predictions.
   (d) Compute the scalar loss with `criterion`.
   (e) Run the backward pass on the loss.
   (f) Step the optimizer.
   (g) Accumulate the loss (weighted by batch size) for the epoch-mean.

3. Return the loss totalled across all examples, divided by the number of examples.

- Return value is a Python `float`, not a tensor.
- Order of operations: zero-grad before forward, step after backward. Reversing these silently breaks training.

5 `validate` (`training.py`)

```
def validate(model: nn.Module, loader: DataLoader, criterion: nn.Module,
    device: torch.device | str) -> tuple[float, float]
```

Evaluate the model on `loader` without updating parameters. Return `(loss, accuracy)` as Python floats.

Algorithm:

1. Switch the model to eval mode (so `Dropout` is the identity and `BatchNorm1d` uses its running statistics).
2. Open a `torch.no_grad()` context (prevents PyTorch from building the computation graph; saves memory and time).
3. For each `(x, y)` batch in `loader`:
   (a) Move the batch tensors onto `device`.
   (b) Feed the batch through the model to obtain logits.
   (c) Compute the loss with `criterion`.
   (d) Take the argmax of the logits along the class dimension to get a predicted class index per example.
   (e) Accumulate the loss (weighted by batch size) and the count of correct predictions.
4. Return the mean loss and the mean accuracy across all examples as a two-tuple.

- Both return values are Python `floats`.
- Accuracy is always in $[0, 1]$.
- The function must **not** modify the model parameters.

6 `train` (`training.py`)

```
def train(model: nn.Module, train_loader: DataLoader, val_loader: DataLoader,
    optimizer: torch.optim.Optimizer, criterion: nn.Module,
    max_epochs: int, patience: int, device: torch.device | str,
    scheduler: torch.optim.lr_scheduler.LRScheduler | None = None) -> dict
```

The full training orchestrator. Runs the train/validate cycle, drives the optional LR scheduler, applies inline early stopping, keeps the best-validation parameters in memory, and restores them before returning.

Algorithm:

1. Initialize:
   - An empty history dict with keys `train_loss`, `val_loss`, `val_acc`, `best_epoch`. The first three start as empty lists; `best_epoch` starts at 0.
   - A best-loss-so-far variable, set to $+\infty$.
   - A deep copy of the current `model.state_dict()` as the best snapshot.
   - An "epochs since improvement" counter, set to 0.
2. For epoch = 0 up to `max_epochs - 1`:
   (a) Call `train_one_epoch`; append the returned loss to `history['train_loss']`.
   (b) Call `validate`; append the loss and accuracy to `history['val_loss']` and `history['val_acc']`.
   (c) If `scheduler is not None`, call `scheduler.step()`.
   (d) If the new val loss is strictly less than the best-so-far: update best, deep-copy `model.state_dict()` as the new best snapshot, set `history['best_epoch']` to the current epoch, reset the counter to 0.
   (e) Otherwise: increment the counter. If it reaches `patience`, break out of the loop (early stop).
3. Restore the best snapshot via `model.load_state_dict(...)`.
4. Return the history dict.

Deep copy matters. `model.state_dict()` returns a *view* of the current parameters; if you assign it directly to your "best" variable, that view will be updated on every subsequent gradient step (meaning "best" tracks the latest, not the best). Use `copy.deepcopy` on the state dict.

- Return type: `dict` with the four keys above. `train_loss`, `val_loss`, `val_acc` are `list[float]` of equal length. `best_epoch` is an `int`.
- If early stopping does not trigger, the lists have length `max_epochs`.
- After `train` returns, the model's parameters reflect the best-validation epoch, not the last epoch.

7 Analysis helpers (`analysis.py`)

`analysis.py` contains two helper functions that the teacher-provided `__main__` block uses to run the ablation study. The `__main__` block itself is released as-is below the marked separator — do not modify it. Match the helper signatures exactly.

7.1 `load_fashion_mnist`

```
def load_fashion_mnist(batch_size:  int = 128)
    -> tuple[DataLoader, DataLoader, DataLoader]
```

Load FashionMNIST and return train, val, and test `DataLoader`s as a three-tuple. This is a utility helper, so the spec names the relevant library entry points directly.

Steps:

1. Build a transform pipeline of `transforms.ToTensor()` followed by `transforms.Normalize((0.2860,), (0.3530,))` (the FashionMNIST channel mean and std).
2. Load `torchvision.datasets.FashionMNIST(root="./data", train=True, download=True, transform=transfo`
3. Compute split sizes: 80% for training, 10% for validation, 10% for test, using integer multiplication on the total length. The remainder (if any) goes to the test split.
4. Build a `torch.Generator().manual_seed(42)` so the split is reproducible across runs.
5. Call `torch.utils.data.random_split` with the three sizes and the generator to split the dataset.
6. Wrap each split in a `DataLoader` with the given `batch_size`. Shuffle the train loader; leave val and test unshuffled. Use `num_workers=0`.

Return order: `(train_loader, val_loader, test_loader)`. The split sizes for the FashionMNIST training set of 60,000 examples are 48000/6000/6000.

7.2 `build_model`

```
def build_model(config:  str)
    -> tuple[nn.Module, torch.optim.Optimizer,
        torch.optim.lr_scheduler.LRScheduler | None]
```

Assemble the model, optimizer, and (optional) scheduler trio for one of the four ablation configurations. The shared architecture is: `MLP(input_dim=784, hidden_dims=[256, 128], output_dim=10, ...)`; constants `INPUT_DIM`, `HIDDEN_DIMS`, `OUTPUT_DIM` are defined at the top of `analysis.py`.

The four valid `config` values and what they each build:

`config`	`dropout_p`	`use_batchnorm`	`weight_decay`	scheduler
`"baseline"`	0.0	False	0	None
`"dropout"`	0.3	False	0	None
`"batchnorm"`	0.0	True	0	None
`"both_schedule"`	0.3	True	10^{-4}	StepLR

For all four configs, use `torch.optim.SGD` with `lr=0.1`. When a scheduler is requested (`"both_schedule"` only), use `torch.optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.5)`. For an unknown `config` string, raise a `ValueError`.

- Return order: `(model, optimizer, scheduler)`, with `scheduler` being `None` when not applicable.
- All three configurations without a scheduler must return `None` for the third element — not a no-op object.

8 How to read the test file

`test_network.py` is organized in three layers — work on them in order:

`@pytest.mark.unit` — one piece at a time (35 pts)

Per-method correctness for `Dropout` and `MLP`. Get these green first; they verify the building blocks of `network.py` independently of any training.

`@pytest.mark.integration` — training pipelines (30 pts)

End-to-end behavior for `train_one_epoch`, `validate`, and `train`. These tests use tiny synthetic datasets so they run in seconds.

`@pytest.mark.analysis` — analysis helpers (35 pts)

Tests for `load_fashion_mnist` (correct splits, shapes, dtypes) and `build_model` (each of the four configs returns the right architecture and optimizer settings). The first run will download Fashion-MNIST (30 MB); subsequent runs use the cache in `./data/`.

Run a single layer with:

```
uv run pytest -m unit          # just unit tests
uv run pytest -m integration   # just integration tests
uv run pytest -m analysis      # just analysis tests
```

Each test class is **all-or-nothing** — every test in a class must pass to earn the points for that class.

9 Restrictions

- **No** `torch.nn.Dropout` anywhere in `network.py`. Implementing dropout from scratch is the substantive learning goal of §2. The `conftest.py` parses your source with `ast` and fails the run if it finds any reference to `nn.Dropout`, `torch.nn.Dropout`, or `from torch.nn import Dropout`.
- **No** `sklearn.neural_network`**.** Build the MLP from `torch.nn` primitives.

You *may* use freely:

- `torch.autograd`, `loss.backward()`, `.requires_grad` — this is the first assignment where these are allowed.
- `nn.Linear`, `nn.ReLU`, `nn.BatchNorm1d`, `nn.Flatten`, `nn.Sequential`.
- `torch.optim.SGD`, `torch.optim.lr_scheduler.StepLR`.
- `torch.utils.data.DataLoader`, `torch.utils.data.random_split`.
- `torchvision.datasets.FashionMNIST`, `torchvision.transforms`.